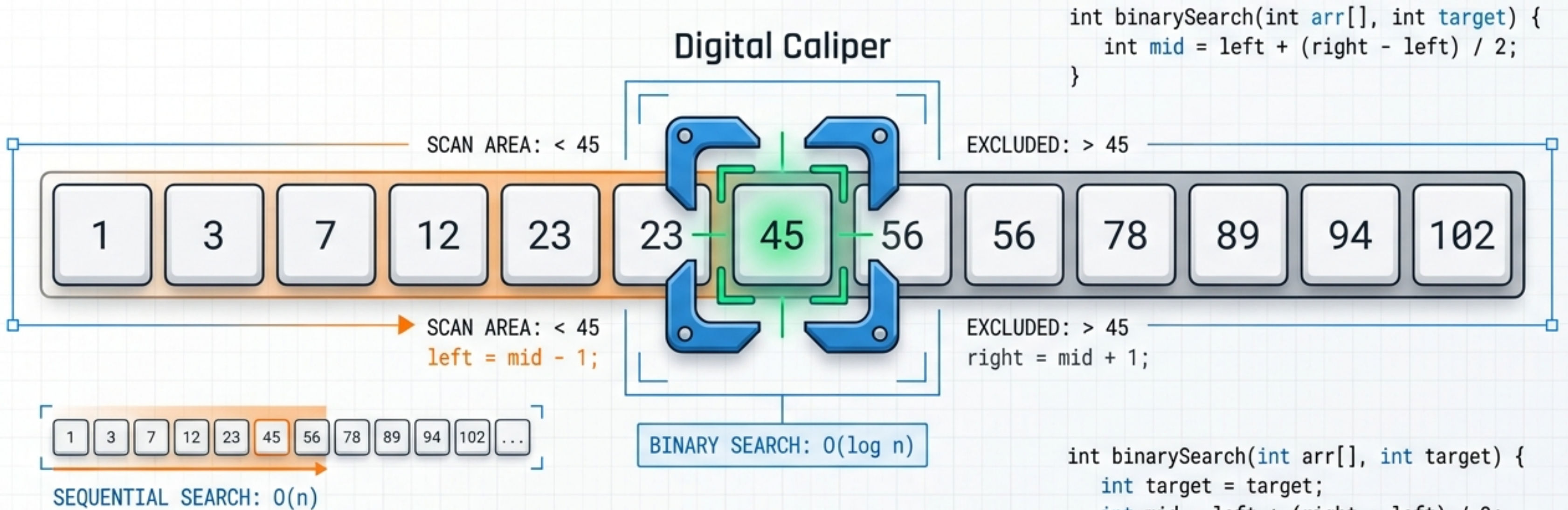


ALGORITMOS DE PESQUISA: SEQUENCIAL E BINÁRIA

Fundamentos de Busca e Complexidade de Desempenho



A Anatomia da Pesquisa

Para medir a eficiência de qualquer algoritmo de busca, precisamos definir a nossa métrica principal.



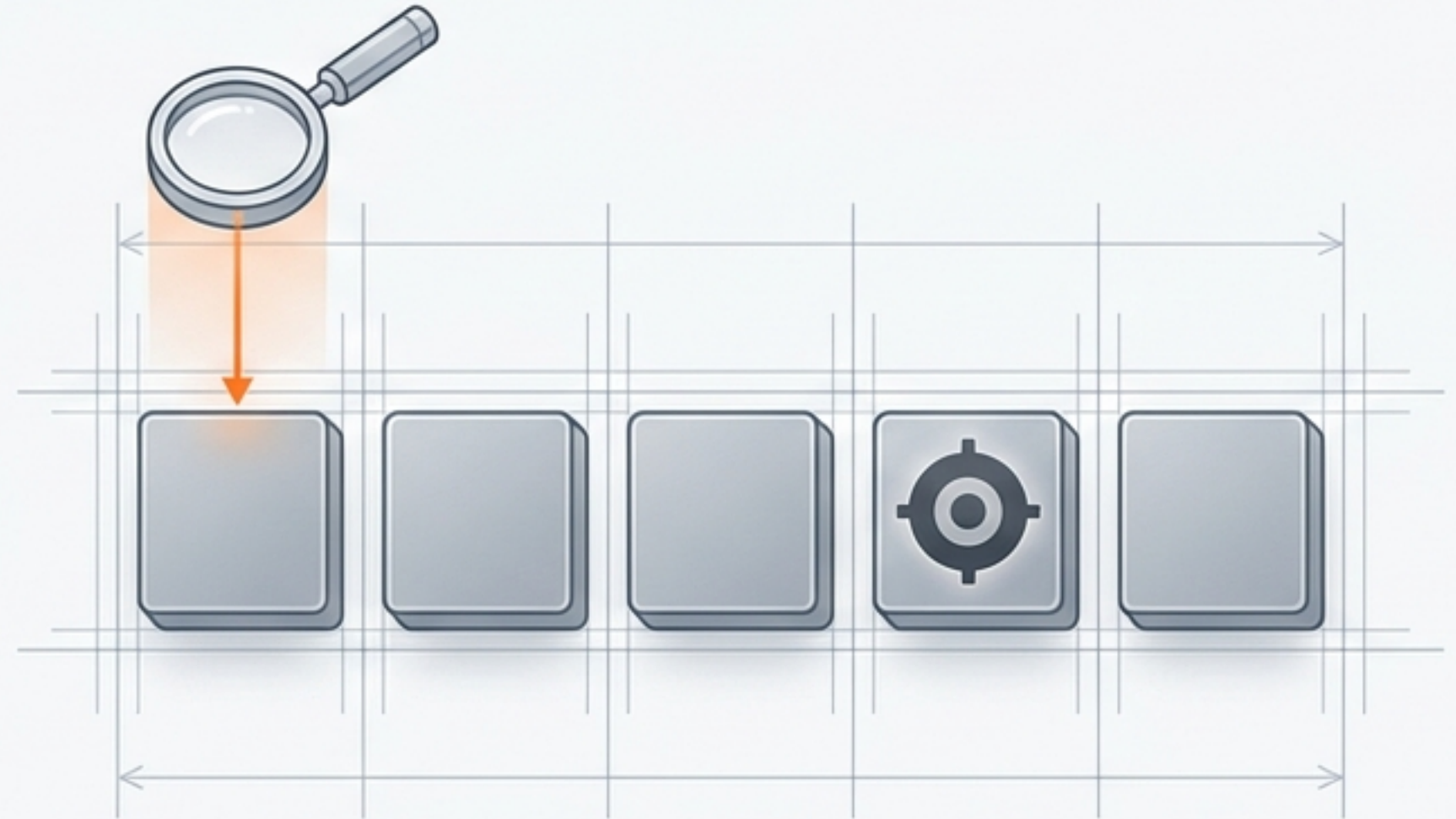
A Operação Relevante:

A comparação entre os elementos do array.



A Pergunta Central:

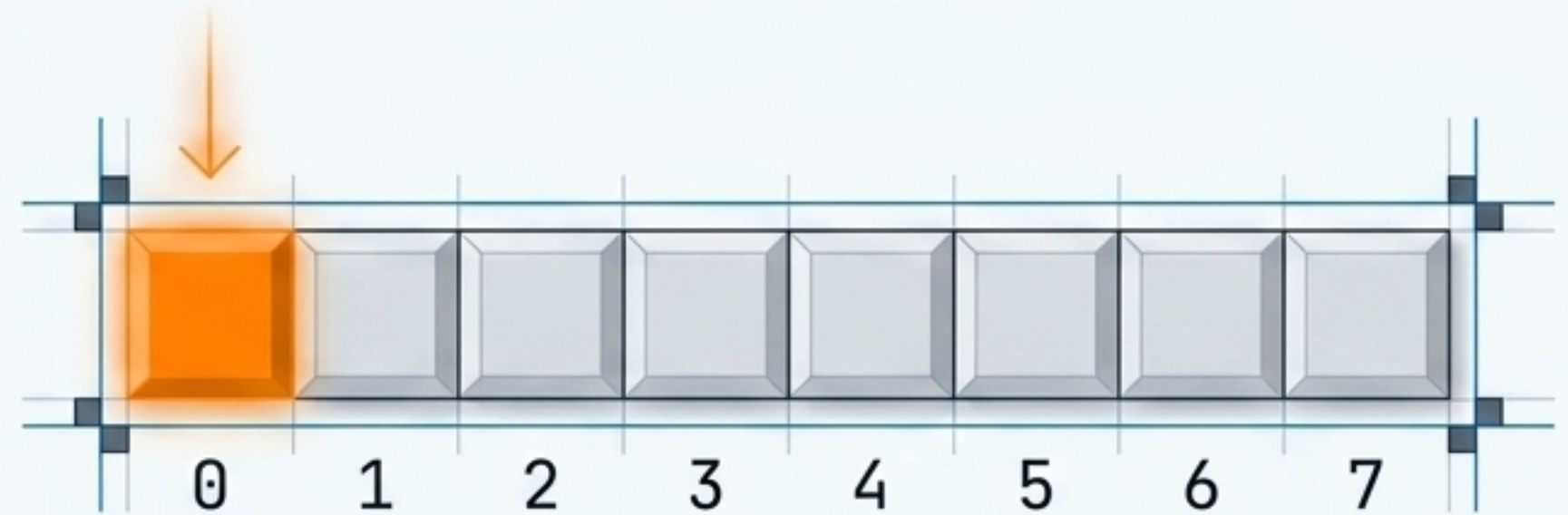
Quantas vezes essa comparação precisará ser executada para encontrar nosso alvo?



Pesquisa Sequencial: O Método da Força Bruta

A lógica padrão: iterar linearmente a partir do índice 0 até $n-1$ até que o elemento seja encontrado.

```
1  boolean resp = false;  
2  for (int i = 0; i < n; i++) {  
3      if (array[i] == x) {  
4          resp = true;  
5          i = n; // break  
6      }  
7  }
```

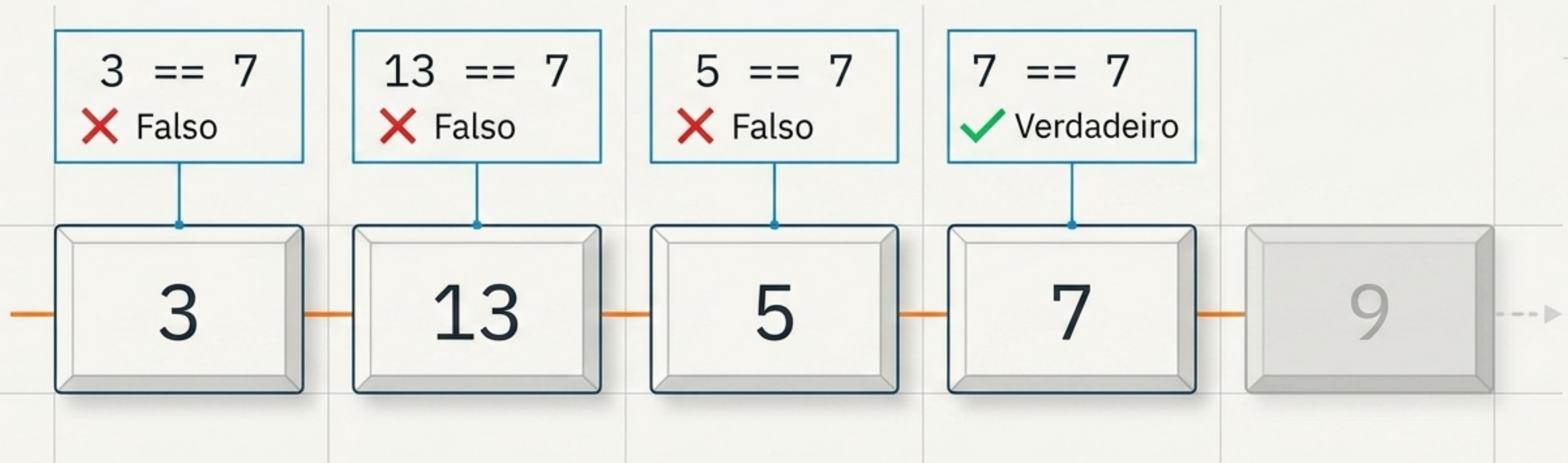


Pesquisa Sequencial em Ação

O loop testa exhaustivamente cada posição. Se o `resp = true`, forçamos a saída definindo `i = n`.

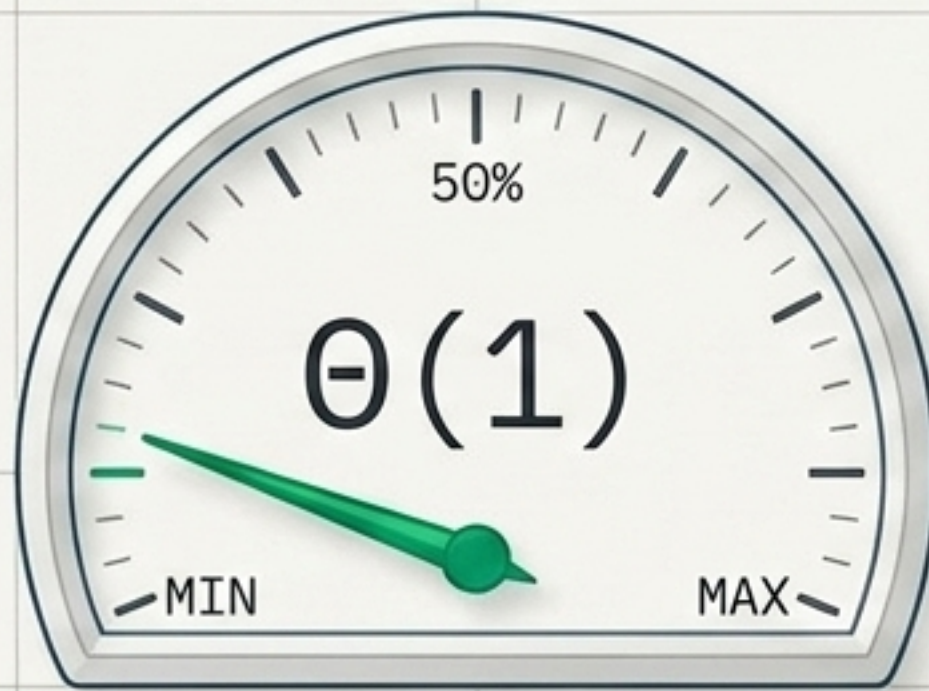
Alvo: 7

Alvo: 7



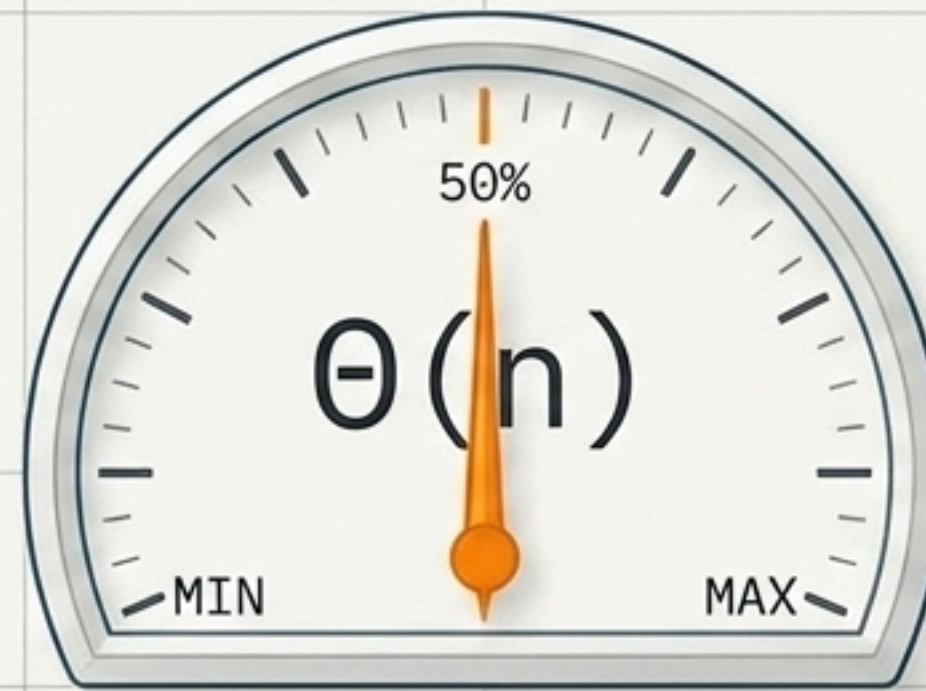
Análise de Desempenho: Sequencial

Melhor Caso



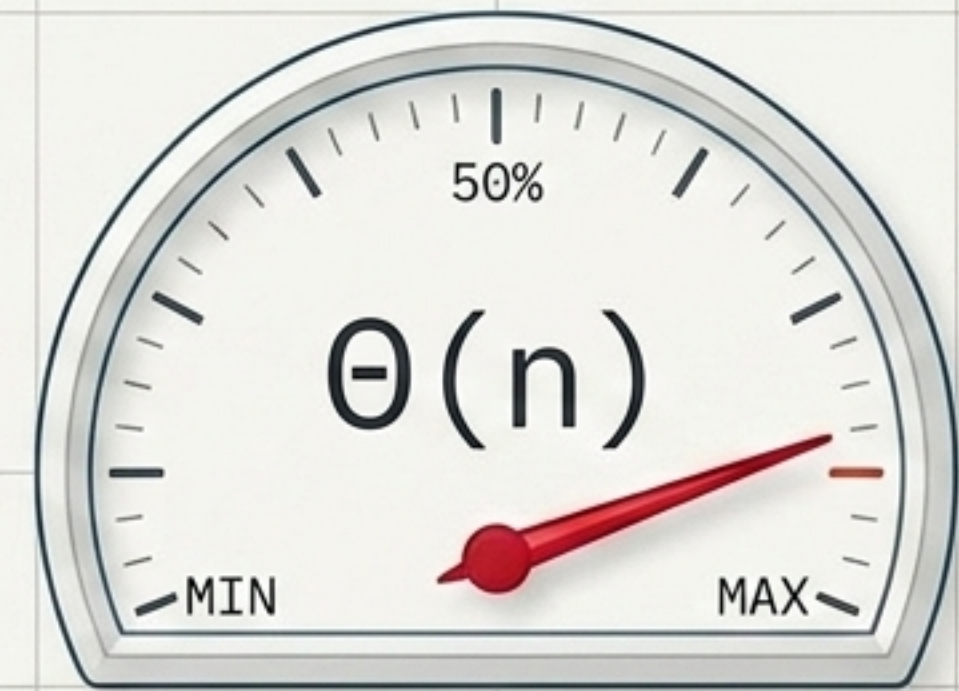
O elemento procurado está exatamente na primeira posição.

Caso Médio



O elemento encontra-se em uma posição intermediária.
 $f(n) = (n+1)/2$.

Pior Caso

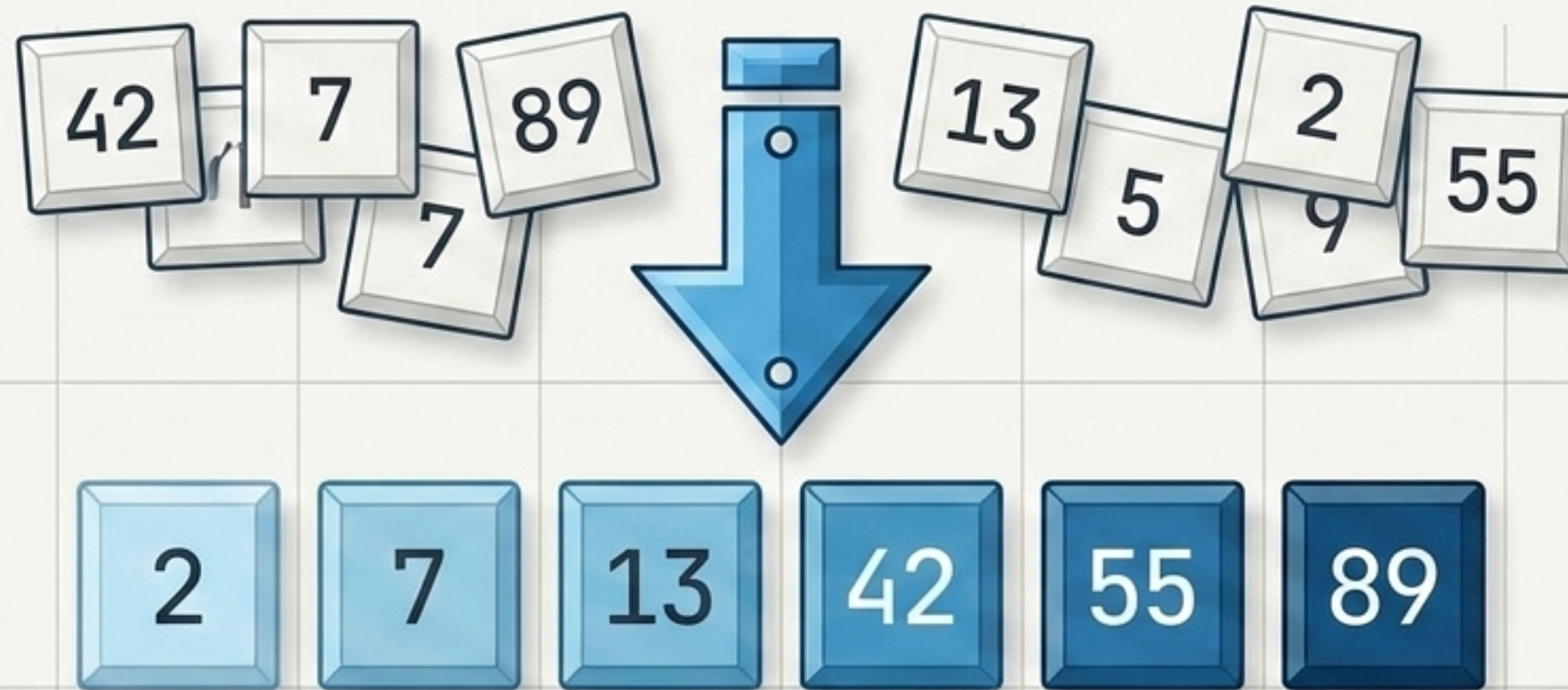


O elemento está na última posição ou simplesmente não existe no array, forçando a varredura completa.

O Ponto de Virada: A Informação Extra

E se tivermos uma informação extra sobre o array? E se ele já estiver ordenado?

A ordenação prévia é o pré-requisito que destrava a possibilidade de um salto de eficiência exponencial. Em vez de caminhar passo a passo, podemos mudar nossa estratégia matemática.



Pesquisa Binária: A Estratégia de Divisão

1. Início ao Centro: Pesquisamos a partir do elemento central do array.
2. Validação Direcional: Se o alvo for maior que a metade, descartamos a metade inferior. Se for menor, descartamos a superior.
3. Repetição: Repetimos o processo exclusivamente com a metade não descartada.



A Mecânica do Código Binário

A condição central de parada é `while (esq <= dir)`. O 'meio' é recalculado dinamicamente a cada iteração.

```
1 int dir = n - 1, esq = 0, meio;  
2 while (esq <= dir) {  
3     meio = (esq + dir) / 2;  
4     if (x == array[meio]) {  
5         resp = true; esq = n;  
6     }  
7     else if (x > array[meio]) {  
8         esq = meio + 1;  
9     }  
10    else {  
11        dir = meio - 1;  
12    }  
13 }
```



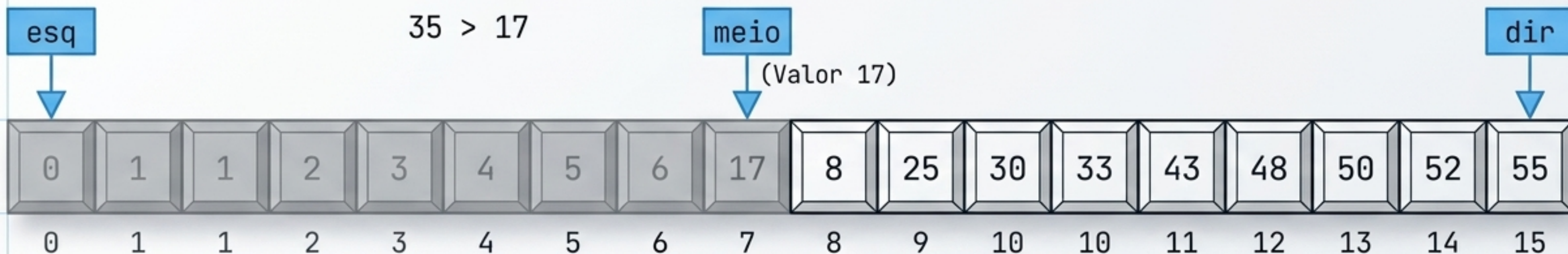
Redução Exponencial em Ação (Buscando 35)

Com uma única comparação, reduzimos o espaço de busca pela metade.

Alvo: 35

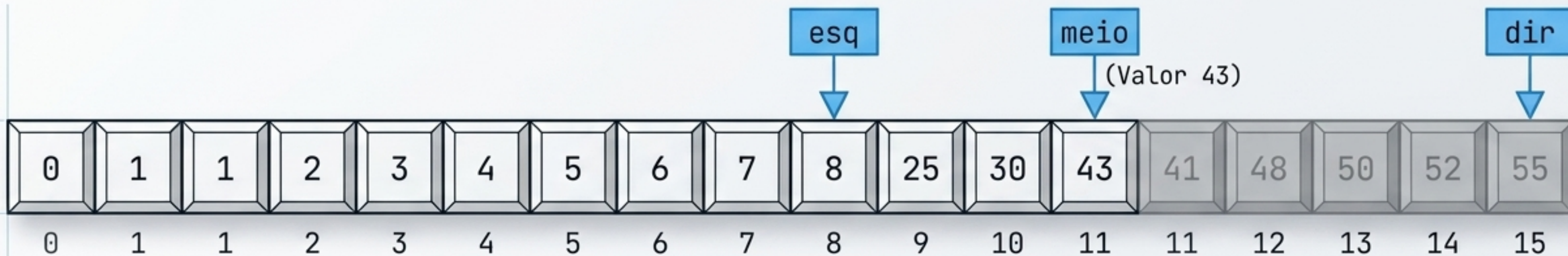
Iteração 1: $esq=0$, $dir=15$, $meio=7$ (Valor 17)

$35 > 17$



$35 < 43$

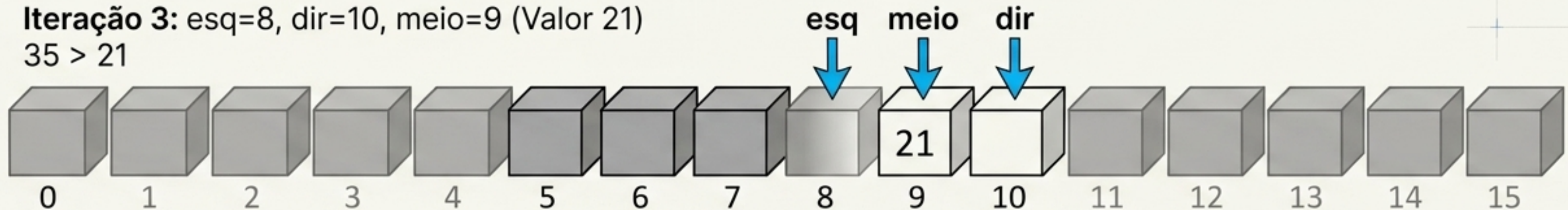
Iteração 2: $esq=8$, $dir=15$, $meio=11$ (Valor 43)



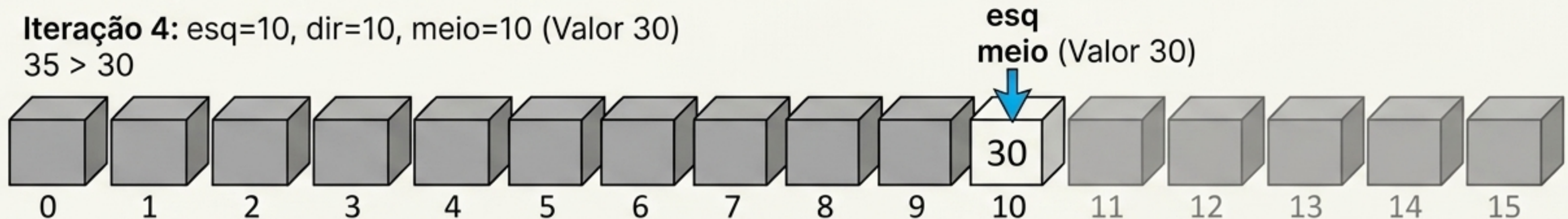
O Encontro e a Condição de Parada

Os ponteiros de limite se cruzam ($esq > dir$). O algoritmo termina com precisão absoluta, confirmando que o 35 não existe no array.

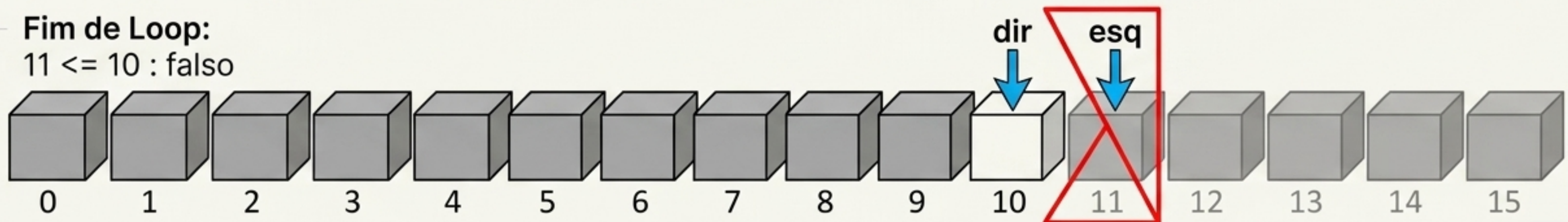
Iteração 3: $esq=8$, $dir=10$, $meio=9$ (Valor 21)
 $35 > 21$



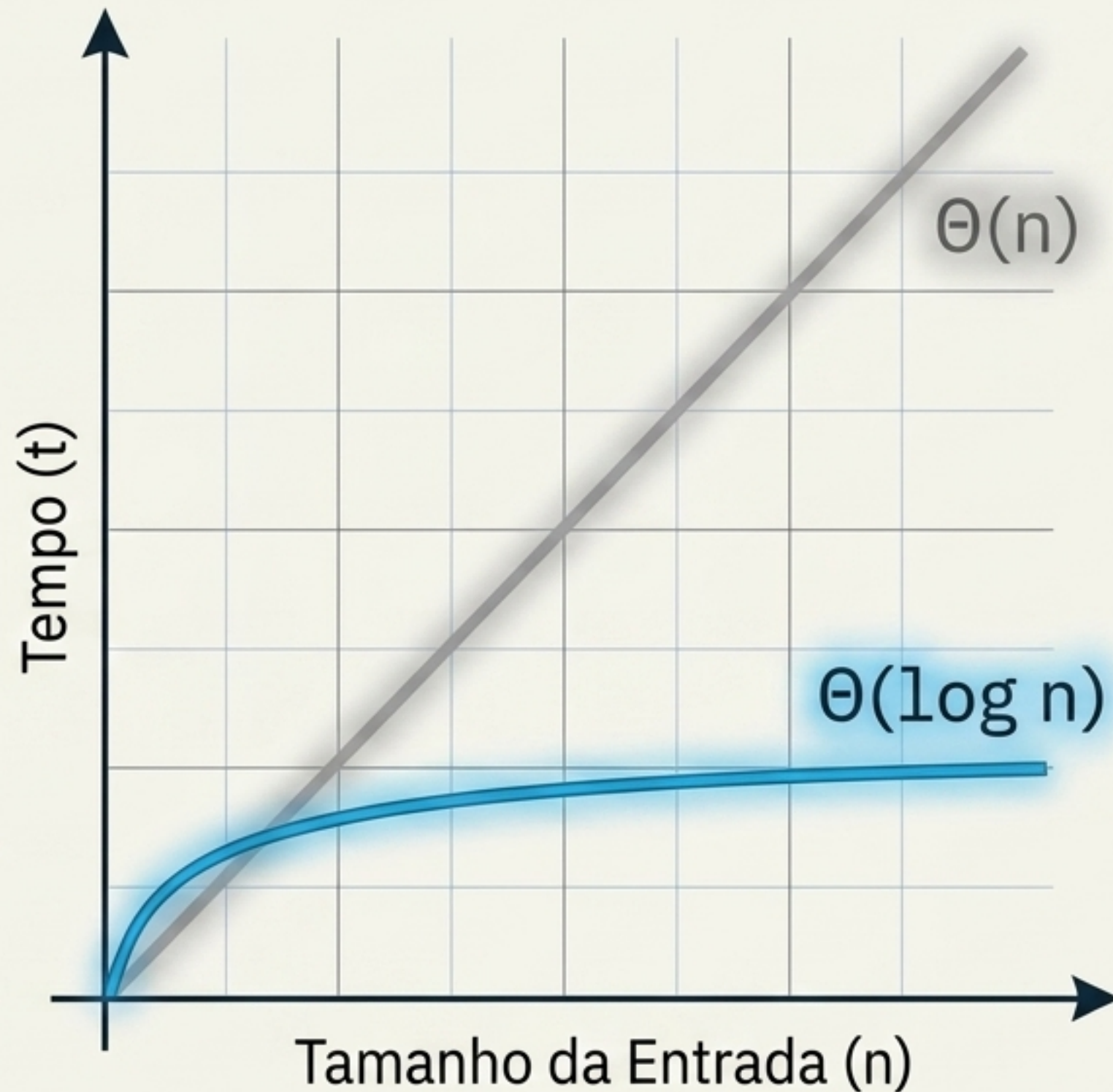
Iteração 4: $esq=10$, $dir=10$, $meio=10$ (Valor 30)
 $35 > 30$



Fim de Loop:
 $11 \leq 10$: falso



Análise de Desempenho: Binária



Melhor Caso: $\Theta(1)$

O elemento procurado está exatamente no centro matemático do *array* logo na primeira verificação.

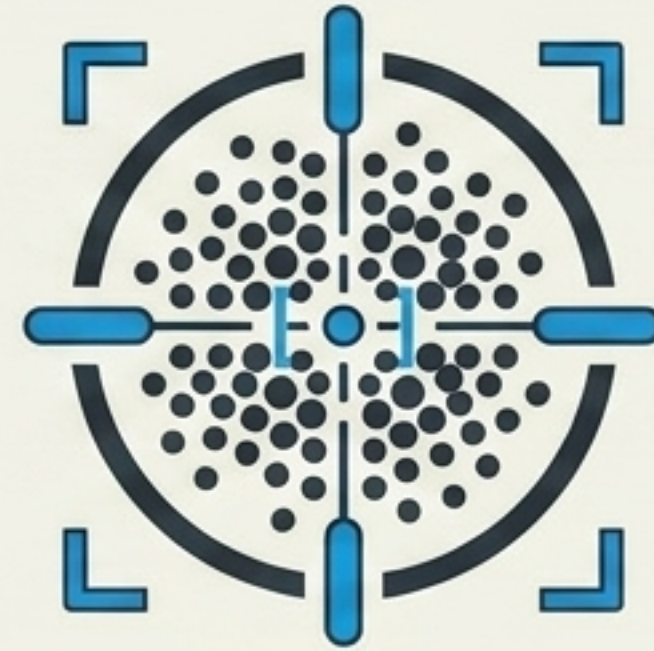
Pior Caso: $\Theta(\log n)$

O elemento está na última posição de procura possível ou não está presente no array. A divisão constante evita o custo linear.

Matriz de Diagnóstico de Algoritmos

	Pesquisa Sequencial	Pesquisa Binária
Pré-requisito	Nenhum	Array Estritamente Ordenado
Mecanismo	Iteração Linear / Varredura	Divisão Logarítmica / Pinça
Melhor Caso	$\Theta(1)$ - Índice 0	$\Theta(1)$ - Índice Central
Pior Caso	$\Theta(n)$	$\Theta(\log n)$

Escalabilidade Requer Estrutura



A Pesquisa Sequencial é flexível e universal, sendo ideal para conjuntos pequenos ou desestruturados. No entanto, o verdadeiro desempenho em larga escala exige a Pesquisa Binária: uma troca onde garantimos a ordem inicial dos dados para conquistar um tempo de busca exponencialmente mais rápido.

A Pesquisa Sequencial é flexível e universal, sendo ideal para conjuntos pequenos ou desestruturados. No entanto, o verdadeiro desempenho em larga escala exige a Pesquisa Binária: uma troca onde garantimos a ordem inicial dos dados conquistar um tempo de busca exponencialmente mais rápido.